Arquitetura do Backend

Checkout de pagamentos com stablecoins

Documento de concepção técnica para Trabalho de Conclusão de Curso (TCC).

Versão 1.0 • Outubro de 2026 • Escopo proposto, sujeito à validação durante o desenvolvimento.

Resumo executivo

Propõe-se um backend desenvolvido em NestJS e TypeScript, organizado em um monorepo com aplicações executáveis independentes e pequenos pacotes de código compartilhado. Internamente, cada aplicação continua dividida em módulos NestJS. Essa combinação permite isolar a entrada de webhooks e o processamento assíncrono, mantendo a lógica de domínio centralizada e evitando a complexidade de microsserviços desnecessários.

O objetivo principal não é prometer disponibilidade absoluta, e sim garantir rastreabilidade, idempotência, recuperação de eventos e consistência do estado de pagamentos mesmo diante de falhas temporárias.

1. Objetivo e escopo

O sistema permitirá que estabelecimentos criem cobranças denominadas em stablecoins, acompanhem o ciclo de vida do pagamento e recebam notificações de mudanças de status. A primeira versão deve operar com uma rede blockchain e um ativo previamente definidos, preferencialmente em ambiente de testes.

- Cadastrar estabelecimentos e credenciais para integração.
- Criar cobranças com identificador único, valor, moeda, rede e prazo de expiração.
- Apresentar instruções de pagamento e registrar transações associadas.
- Verificar eventos de blockchain e número de confirmações conforme a política adotada.
- Atualizar estados do pagamento de maneira idempotente.
- Entregar webhooks assinados aos estabelecimentos, com retentativas e auditoria.
- Executar reconciliação para detectar pagamentos não capturados pelos eventos em tempo real.

Fora do escopo inicial: custódia de fundos, conversão cambial automática, operação multichain e disponibilidade geográfica de nível bancário. Esses itens podem constituir evoluções futuras.

2. Decisão arquitetural

Adota-se um monorepo com três aplicações potenciais: API principal, receptor de eventos (webhook ingress) e worker. A separação operacional pode ser introduzida em etapas; módulos e pacotes não são mecanismos de alta disponibilidade por si só.

Conceito	Papel na solução
Módulo NestJS	Organiza controllers, services e providers dentro de uma

	aplicação.
Pacote	Fornece contratos e bibliotecas reutilizáveis, com dependências explícitas.
Aplicação (app)	Ponto de entrada executável que pode ter container e deploy próprios.
Container	Instância operacional de uma imagem Docker; pode falhar isoladamente de outros processos.

3. Estrutura proposta do repositório

```
stablecoin-checkout/
├── apps/
│   ├── api/              # API pública, merchants e cobranças
│   ├── webhook-ingress/  # Recepção de eventos do provedor
│   └── worker/           # Confirmações, reconciliação e entregas
├── packages/
│   ├── contracts/        # Enums, DTOs de eventos e tipos compartilhados
│   ├── payment-domain/   # Regras puras, estados e validações
│   └── blockchain-client/ # Adaptador reutilizável do provedor
├── infra/
│   ├── docker/
│   └── nginx/
├── pnpm-workspace.yaml
└── package.json
```

Cada app pode conter módulos como AuthModule, MerchantModule, PaymentModule, WebhookModule e ReconciliationModule. Evitar transformar automaticamente todos os módulos em pacotes: extrair somente o código que possui consumidores reais em mais de uma aplicação ou exige fronteira de dependência explícita.

4. Componentes e responsabilidades

Componente	Responsabilidade	Ponto de atenção
API principal	Autenticação, cadastro de merchants, criação e consulta de cobranças.	Sem chave privada de carteira no código.
Webhook ingress	Recebe eventos externos, valida assinatura e persiste antes de responder 2xx.	Curto, idempotente e tolerante a duplicatas.
Worker	Processa eventos, verifica transações e confirmações; entrega webhooks de saída.	Retentativas com backoff e DLQ.
PostgreSQL	Registra cobranças, transações, eventos recebidos e entregas.	Integridade, índices e transações.
Fila persistente	Desacopla recepção de processamento, quando adotada.	Durabilidade e confirmação de consumo.
Nginx	TLS, roteamento reverso e distribuição entre instâncias disponíveis.	Não executa deploy por conta própria.

5. Fluxo principal de uma cobrança

1. O estabelecimento autenticado solicita POST /payments com valor, ativo e referência externa.
2. A API valida os dados, gera paymentId e grava a cobrança como PENDING.
3. A API devolve instruções de pagamento (rede, endereço, valor e expiração) de acordo com o modelo de recebimento definido.
4. Um provedor ou monitor on-chain identifica uma transação candidata e gera um evento.
5. O webhook ingress valida a autenticidade do evento e registra seu identificador em armazenamento durável antes de confirmar o recebimento.
6. O worker carrega a cobrança, valida rede, token, endereço, valor, hash, destinatário e política de confirmações; então atualiza o estado por transação atômica.
7. A alteração de estado cria um evento de saída persistido para notificar o merchant; uma rotina entrega o webhook e registra tentativas.

```
POST /payments  ->  PENDING
Evento on-chain ->  DETECTED  ->  CONFIRMING  ->  CONFIRMED
                          \->  REVIEW_REQUIRED (divergências)
Tempo expirado ->  EXPIRED (sem confirmação elegível)
```

O modelo de estados é ilustrativo. A política deve especificar casos de pagamento parcial, valor excedente, expiração durante confirmações e eventuais reorganizações da blockchain; uma cobrança expirada não implica ignorar fundos recebidos posteriormente.

6. Confiabilidade e idempotência

Eventos podem chegar duplicados, fora de ordem ou não chegar. A confiabilidade não deve depender de um único webhook. Recomenda-se:

- Chave única do evento do provedor para impedir reprocessamento indevido.
- Restrição única apropriada para rede + hash de transação + índice de log quando aplicável.
- Máquina de estados com transições válidas e atualização transacional.
- Padrão inbox: guardar eventos recebidos antes de dar sucesso ao emissor; em alternativa, publicar em fila durável com confirmação apropriada.
- Padrão outbox para registrar a intenção de notificar o merchant na mesma transação que muda o pagamento.
- Retentativas com backoff, limite de tentativas, dead-letter queue e operação de reprocessamento.
- Reconciliação periódica consultando a rede/provedor como segunda fonte de detecção.

“Exactly-once delivery” não é assumido. A meta prática é entrega ao menos uma vez combinada com processamento idempotente.

7. Dados e contratos principais

Entidade	Campos mínimos sugeridos
merchants	id, name, status, created_at
payments	id, merchant_id, amount, asset, network, destination, status, expires_at
blockchain_transactions	id, network, tx_hash, log_index, block_number, confirmations, payment_id

inbound_events	id, provider, provider_event_id, received_at, payload_ref, processed_at, status
outbox_events	id, event_type, aggregate_id, payload, created_at, published_at
webhook_deliveries	id, event_id, merchant_id, attempt, response_code, next_retry_at, status

O formato numérico de quantias on-chain deve evitar ponto flutuante: armazenar unidades mínimas com inteiro/decimal adequado e conservar a precisão do token. Segredos e informações sensíveis não devem ser incluídos em logs.

8. Segurança

- HTTPS obrigatório em interfaces externas; certificados administrados no proxy/infraestrutura.
- Autenticação e autorização por merchant; segregação dos dados por estabelecimento.
- Validação HMAC ou mecanismo oficial do provedor para webhooks de entrada, usando corpo bruto quando exigido.
- Webhooks de saída assinados com segredo por merchant e timestamp; proteção contra replay.
- Segredos em variáveis/gerenciador de segredos, nunca versionados no Git.
- Rate limit, validação de DTOs, logs de auditoria e controle de acesso administrativo.
- Confirmação on-chain antes de marcar como CONFIRMED; não confiar apenas no conteúdo de um webhook.

9. Infraestrutura, Docker e deploy

Cada aplicação executável pode gerar sua própria imagem Docker. Uma imagem serve de artefato imutável de deploy; pacotes compartilhados são compilados como dependências das aplicações consumidoras. PostgreSQL e fila devem ter persistência e rotina de backup próprias.

```
Repositório -> CI (testes/build) -> Imagens versionadas -> Registry
                                        |
                                Deploy API / Webhook / Worker
                                        |
                                    Nginx / Containers
```

Para reduzir downtime do webhook ingress: iniciar a nova instância, checar readiness, alternar o tráfego no Nginx ou balanceador, aguardar requisições antigas e encerrar a instância anterior graciosamente. Uma VPS única continua sendo um ponto de falha, mesmo com containers separados.

10. Estratégia de implementação do TCC

Etapa	Entrega	Critério de validação
MVP 1	API modular, PostgreSQL, cobranças e integração testnet.	Criar e consultar cobrança corretamente.
MVP 2	Recepção de eventos, verificação on-chain, idempotência.	Evento duplicado não duplica confirmação.
MVP 3	Worker/fila e entrega confiável de webhooks.	Retentativa recupera falha temporária.

MVP 4	Reconciliação, Docker, health check e testes de falha.	Pagamento detectado mesmo após perda simulada de webhook.

11. Critérios de qualidade e validação

- Testes unitários das transições de estado, precisão de valores e assinatura HMAC.
- Testes de integração de API, banco e consumidor de eventos.
- Teste de duplicidade: enviar o mesmo evento várias vezes e verificar atualização única.
- Teste de indisponibilidade: derrubar worker e comprovar reprocessamento posterior.
- Teste de reconciliação: simular webhook perdido e recuperar transação pela fonte on-chain.
- Teste de deploy: manter endpoint de entrada disponível durante troca controlada de instâncias, quando viável na infraestrutura.

12. Conclusão e decisão recomendada

A arquitetura recomendada combina módulos NestJS para organização interna, pacotes restritos a código realmente compartilhado e aplicações separadas apenas onde há necessidade operacional. O foco técnico do TCC deve ser o ciclo confiável de pagamentos em stablecoins: receber, verificar, persistir, reconciliar e notificar. Disponibilidade é importante, mas deve ser acompanhada de recuperação e idempotência para preservar a consistência financeira.

Próxima decisão de projeto: definir blockchain, stablecoin, modelo de endereçamento de recebimento (por cobrança ou compartilhado) e provedor de observação on-chain. Essas escolhas afetam diretamente as regras de confirmação e a modelagem dos dados.